



JavaPhone™ API Specification

Version 1.0

March 22, 2000

Copyright © 1999-2000 Sun Microsystems, Inc.
901 San Antonio Road, Palo Alto, CA 94303 USA

All rights reserved.

This product or document is protected by copyright and distributed under licenses restricting its use, copying, distribution, and decompilation. No part of this product or related documentation may be reproduced in any form by any means without prior written authorization of Sun and its licensors, if any. Third party software, including font technology, is copyrighted and licensed from Sun suppliers.

Sun, Sun Microsystems, the Sun Logo, JDK, SunXTL, the Duke logo, Java, JavaPhone, and all Java-based marks, are trademarks or registered trademarks of Sun Microsystems, Inc in the U.S. and other countries.

PostScript is a registered trademark of Adobe Systems, Inc.

Federal Acquisitions: Commercial Software--Government Users Subject to Standard License Terms and Conditions.

The OPEN LOOK(R) and Sun™ Graphical User Interfaces were developed by Sun Microsystems, Inc. for its users and licensees. Sun acknowledges the pioneering efforts of Xerox in researching and developing the concept of visual or graphical user interfaces for the computer industry. Sun holds a non-exclusive license from Xerox to the Xerox Graphical User Interface, which license also covers Sun's licensees who implement OPEN LOOK GUIs and otherwise comply with Sun's written license agreements.

DOCUMENTATION IS PROVIDED "AS IS" AND ALL EXPRESS OR IMPLIED CONDITIONS, REPRESENTATIONS AND WARRANTIES, INCLUDING ANY IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE OR NON-INFRINGEMENT, ARE DISCLAIMED, EXCEPT TO THE EXTENT THAT SUCH DISCLAIMERS ARE HELD TO BE LEGALLY INVALID.

Copyright 1999 Sun Microsystems, Inc.,
901 San Antonio Road, Palo Alto, Californie, 94303, Etats-Unis.

Tous droits réservés.

Ce produit ou document est protégé par un copyright et distribué avec des licences qui en restreignent l'utilisation, la copie, la distribution, et la décompilation. Aucune partie de ce produit ou document ne peut être reproduite sous aucune forme, par quelque moyen que ce soit, sans l'autorisation préalable et écrite de Sun et de ses bailleurs de licence, s'il y en a. Le logiciel détenu par des tiers, et qui comprend la technologie relative aux polices de caractères, est protégé par un copyright et licencié par des fournisseurs de Sun.

Sun, Sun Microsystems, le logo Sun, JavaPhone(TM), JDK, SunXTL, le logo Duke, JavaPhone, et Java(TM) sont des marques de fabrique ou des marques déposées de Sun Microsystems, Inc. aux Etats-Unis et dans d'autres pays.

Postscript est une marque enregistrée d'Adobe Systems Inc.

Le interface d'utilisation graphique OPEN LOOK (R) et Sun (TM) a été développée par Sun Microsystems, Inc. pour ses utilisateurs et licenciés. Sun reconnaît les efforts de pionniers de Xerox pour la recherche et le développement du concept des interfaces d'utilisation visuelle ou graphique pour l'industrie de l'informatique. Sun détient une licence non exclusive de Xerox sur l'interface d'utilisation graphique Xerox, cette licence couvrant également les licenciés de Sun qui mettent en place l'interface d'utilisation graphique OPEN LOOK et qui en outre se conforment aux licences écrites de Sun.

LA DOCUMENTATION EST FOURNIE "EN L'ETAT" ET TOUTES AUTRES CONDITIONS, DÉCLARATIONS ET GARANTIES EXPRESSES OU TACITES SONT FORMELLEMENT EXCLUES DANS LA MESURE AUTORISÉE PAR LA LOI APPLICABLE, Y COMPRIS NOTAMMENT TOUTE GARANTIE IMPLICITE RELATIVE À LA QUALITÉ MARCHANDE, À L'APTITUDE À UNE UTILISATION PARTICULIÈRE OU À L'ABSENCE DE CONTREFAÇON.

For further information on Intellectual Property matters, contact Sun's Legal Department:
E-Mail: trademarks@sun.com
Phone: 650.960.1300

JavaPhone™ Overview

The JavaPhone API is a vertical extension to the PersonalJava™ platform developed through an open process by Sun Microsystems in collaboration with other key expert groups in the telecommunications engineering community. This API, combined with the PersonalJava API, provides the ideal development and deployment platform for telephony applications and services targeted at wireless communicators and smart phones, as well as Internet screenphones.

The JavaPhone API extends the PersonalJava platform by adding access to functionality unique to client telephony devices as follows:

- Direct telephony control
- Datagram messaging
- Access to address book information
- Access to calendar information
- Access to user information
- Power management
- Application installation mechanisms

In addition to the core API interfaces that comprise the JavaPhone platform, the specification also describes the Communication API and the Secure Sockets Layer (SSL) API, which are optional in the PersonalJava API, but may be required when operating phone terminals in specific environments.

The JavaPhone API can meet the telephony needs of the following users:

- Wireless device manufacturers
- Network operators
- Application developers

Wireless device manufacturers can bring telephony products to market more rapidly by taking advantage of platform-independent, standard software components. New wireless phones or Internet screenphones that feature a JavaPhone API implementation can leverage positioning in the marketplace by taking advantage of an ever increasing number of industry standard compliant applications and services.

Network operators benefit from the ability to deliver value-added business solutions for a diverse array of Java technology-enabled devices in their networks regardless of the underlying microprocessor or operating system on the device. This group of telephony users can maintain and grow their competitive advantage by upgrading and managing their phone services dynamically, allowing subscribers to have access to new types of services and applications in a transparent fashion. In essence, value-added services

written in the Java programming language offer security, extensibility, and platform independence.

Application developers can quickly develop cost-effective service applications while ensuring portability across a wide variety of devices. In addition, this group benefits from the ability to utilize a true object-oriented language that allows them to save time and money by reusing robust software components.

The JavaPhone API in concert with the PersonalJava platform provides an ideal development environment for an emerging class of value-added services that can be deployed across a diverse array of Java technology-enabled devices.

PersonalJava Platform and JavaPhone

The JavaPhone APIs are a vertical extension that run on implementations of the PersonalJava 1.1.1 specification, that specifies required, optional, and modified behavior based on the JAE (Java Application Environment) 1.1 specification. Some JavaPhone API device implementations may require interfaces that are defined in the PersonalJava 1.1.1 specification as optional packages.

JavaPhone Interfaces

The JavaPhone interfaces were developed by the JavaPhone team in conjunction with other telephony and mobile computer forums using Sun Microsystems' open process. Collectively, these interfaces meet the requirements of wireless and screenphone based devices.

Applications may be designed to take advantage of specific API features in the JavaPhone interfaces to support a select category of devices. Alternatively, applications can be written to leverage the more common required packages available on all devices to reach the largest possible installed customer base.

A profile is a collection of one or more Java APIs supported on a Java consumer platform that provide an ideal development and deployment environment for a specific device type. JavaPhone API requirements for both wireless and screenphone profiles are described in the following table.

JavaPhone API	Wireless Profile	Screenphone Profile
<u>Addressbook</u>	Required	Required
<u>User Profile</u>	Required	Required
<u>JTAPI Core</u>	Required	Required
<u>JTAPI Mobile</u>	Required	Optional
<u>Calendar</u>	Required	Optional
<u>Network Datagram</u>	Required for connectionless transport	Optional
<u>Power Monitor</u>	Required	Optional
<u>Power Management</u>	Optional	Optional
<u>Install</u>	Optional	Optional

Optional PersonalJava Interfaces

The Communication API and the SSL API are optional interfaces in the PersonalJava API specification but may be required in JavaPhone for devices that support certain network and local device capabilities.

API	Required	Comments
<u>Communication API</u>	Required for serial ports	Must allow access to IR port
<u>SSL</u>	Required for secure socket communications	The <code>https</code> protocol may be used without explicitly supporting <code>javax.net.ssl</code>

Security Recommendations for JavaPhone Interfaces

In some environments, it is desirable to control access to confidential data. Various components of JavaPhone interfaces can contain valuable and sensitive information that could be exploited by a dysfunctional application or applet. To protect device data, only trusted applications should be able to retrieve information from the JavaPhone interfaces.

Untrusted code should not be allowed to perform the following events:

- Read or write data to databases
- Retrieve calendar or address book information
- Receive or send datagrams

Java™ Telephony API (JTAPI) Core Package

Java™ Telephony API (JTAPI) is designed to fit a variety of telephony application configurations using a core package of interfaces that define the basic call model and a variety of extensions (optional packages) that build on the basic foundation classes. Some of these extensions are also included in the JavaPhone API. The core interfaces from JTAPI provide the framework for using the telephony functions of the device and allow ISV and third-party applications to make and receive calls, as well as monitor call progress.

In addition to the JTAPI functionality partitioned into separable packages, a capabilities package serves as an extension mechanism that supports specific named *capabilities*. A JTAPI Provider announces to applications the features included within a specific vendor's implementation using named capabilities.

For more information, refer to the following:

- [JTAPI Core API Specification](#)
- [JTAPI overview and product information](#)
- [JTAPI 1.2 API Specification and White Paper](#)
- [JTAPI 1.3 Early Access Public Review Specification](#)

JTAPI Package Specifications

The following JTAPI package specifications are supported in the JavaPhone 1.0 specification. (Note that each JTAPI package includes a subdirectory for events and a subdirectory for capabilities interfaces.)

API	Wireless Profile	ScreenPhone Profile
JTAPI Core Package	Required	Required
JTAPI Call Control Package	Optional	TBD
JTAPI Phone Package	Optional	TBD

JTAPI Core Package

The JTAPI Core package meets the requirement for basic call management: *make*, *answer* and *drop call(s)*. The interfaces in the JTAPI core package also have specific extensions in the JTAPI Mobile package to meet the requirements of specific mobile (wireless) phone network capabilities.

For more information on the relationship between the JTAPI Core package and the standard extensions, refer to the JTAPI 1.2 Overview.

The following specifications describe the interfaces included in the JTAPI Core package that are required for all JavaPhone devices.

- [javax.telephony](#)
- [javax.telephony.events](#)
- [javax.telephony.capabilities](#)

JTAPI Call Control Package

The JTAPI Call Control package provides extended features for conferencing and forwarding calls. This package is **optional** for JavaPhone devices. For more information on the JTAPI Call Control interface definitions, refer to the JTAPI 1.2 Call Control Extension Package Overview.

The following specifications describe the interfaces in the JTAPI Call Control package:

- [javax.telephony.callcontrol](#)
- [javax.telephony.callcontrol.events](#)
- [javax.telephony.callcontrol.capabilities](#)

JTAPI Phone Package

The JTAPI Phone package provides methods for defining the physical characteristics of the phone, such as ringer settings, switchhook, or buttons. This package is **optional** for JavaPhone devices.

The Phone package may be necessary, for example, if the physical components of a phone terminal are used in an application that manipulates the typical phone functions such as tracking the number of button entries necessary to record numbers for speed dialing. For more information on the Phone package architecture, refer to the JTAPI 1.2 Phone Package Overview.

The following specifications describe the interfaces in the JTAPI Phone package:

- [javax.telephony.phone](#)
- [javax.telephony.phone.events](#)
- [javax.telephony.phone.capabilities](#)

JTAPI Mobile Package

The core JTAPI interfaces are designed to support capability interfaces that declare the type of services an application developer can rely on in the platform. However, the space constraints of a mobile telephony device cannot afford the substantial degree of overhead to simply designate whether or not a feature is supported on the device

In the JTAPI mobile package, groups of related features have been placed into an interface to allow individual implementations to decide whether those particular features will be provided on their platform. For example, the application developer may test the MobileProvider instance to determine if it is an instance of MobileRadio before attempting to offer a user interface that controls the startup state of the radio hardware.

- JTAPI Mobile API Specification

API	Wireless Profile	ScreenPhone Profile
JTAPI Mobile package	Required	TBD

The following JTAPI Mobile core interfaces are included in the JavaPhone 1.0 Specification.

JTAPI Mobile core interfaces

Interface	Optional/Required	Capability
<u>MobileProvider</u> <u>MobileProviderEvent</u> <u>MobileProviderListener</u>	Required	These are the key interfaces for JTAPI Mobile and define the relationship between network, subscription, and JTAPI. They provide a basic listener/event mechanism for monitoring changes in the current telephony implementation.
<u>MobileAddress</u>	Required	Mobile extension of the core Address for subscription Id.
<u>MobileTerminal</u>	Required	Mobile extension of the core Terminal interface.
<u>MobileNetwork</u>	Required	Basic information about recognizable mobile networks
<u>NetworkSelection</u>	Optional	GSM specific forbidden and preferred network list selection mechanism
<u>MobileRadio</u> <u>MobileRadioEvent</u> <u>MobileRadioListener</u>	Optional	Signal strength and startup state management interfaces of the radio device

AddressBook

The AddressBook API provides applications with access contact information. Leveraging the vCard specification (RFC 2426), the AddressBook API provides storage for and access to business card information. An application using the AddressBook API can locate and update contact information such as phone numbers, postal addresses, email addresses, and other information about individuals, groups, and organizations. In GSM phones, for example, the AddressBook API provides access to name and address information stored on the SIM card.

The primary design goal of the AddressBook API is to be simple and thin so that it can fit in resource-limited devices. Because the AddressBook API is an extensible API that leverages a standard schema, application developers can easily find and use contact information for writing programs. The API specifies a base storage class for all types of items including contact items, calendar entries, and todo entries. Schema strings used in querying and storing contact information are those specified in vCard. (vCard personal data interchange format has been accepted as a standard method to transfer contact

information.)

- [AddressBook API Specification](#)
- [RFC 2426: vCard MIME Directory Profile](#)

User Profile

The User Profile API provides an application with information about the current user or owner of the device. For example, in a GSM phone, the User Profile API provides access to the SIM card information for the owner which might include the user's name and other attributes that identify the user.

The current user profile information is returned in the same format as the AddressBook API.

- [User Profile API Specification](#)

Calendar

The Calendar API provides applications with access to schedule information. Leveraging the iCalendar specification (RFC 2445), the Calendar API provides storage for and access to schedule information such as dates and task entries. CalendarEntries provide access to descriptions, dates, and repeat information. ToDo entries provide for access to notes and items to be done. Scheduling data stored in a device implemented with the Calendar API is accessible to a user who can easily locate dates and times or make modifications to this information.

The primary design goal of the Calendar API is to be simple and thin so that it can fit in resource-limited devices. The API specifies a base storage class for all types of items including calendar entries and todo entries. Schema strings used in querying and storing contact information are those specified in vCalendar. (vCalendar interchange format has been accepted as a standard way to transfer scheduling information.

- [Calendar API Specification](#)
- [RFC 2445: Internet Calendaring and Scheduling Core Object Specification](#)

Network Datagram

The Network Datagram API provides for transport independent addressing and delivery of messages. Applications send and receive messages using addresses consisting of service name and service location. The API allows applications to be developed independently of the physical network or bearer supported by the device. The physical network or bearer used to deliver the message is selected by the device when the message is sent. For example, on a wireless phone the GSM short message service (SMS) can be used.

The API allows application developers to know only the addressing and data protocol while hiding the details of the transport. Therefore, applications do not have to be rewritten to run on different phones or networks.

The Network Diagram API is capable of addressing and delivering messages on transports (bearers) common to wireless devices. The following are examples of wireless device messaging:

- Recipients (A named server and service)
- WAP datagrams
- SMS messages

The API supports one or more applications and transport(s) concurrently in a single VM (Virtual Machine):

- SMS as bearer for SMS message
- SMS as bearer for WDP datagrams
- TCP/IP Data as transport for UDP
- [Datagram API Specification](#)

Install

The JavaPhone extension includes an interface to install and remove applications. To support JavaPhone application development and deployment, the API supports the following installation functions:

- Application packaging for development and distribution, including the following components:
 - Jar files that contain application classes
 - Unique identification of an application
 - Identifying the main entry point class for an application
 - Identifying icons to represent the application in a user interface
- Installation mechanisms for installing an application into the environment
- Versioning - identifying what version of an application is present
- [Install API Specification](#)

Power Monitor

The Power Monitor API monitors the available power levels of the device and notifies when the device is changing power states. Using the Power Monitor API, the application can tailor its activities to the available power. The Power Monitor API allows an application to determine the state of power on the device. It allows an application to retrieve the current battery level, an estimate of remaining battery life, and whether an external power source is being used. The API also provides notification when battery power is running low and a service is about to be terminated as a result.

- [Power Monitor API Specification](#)

Power Management

Applications can use the Power Management API to determine the power state of the device. For example, the ON power state can be identified. The API allows an application to be aware of transitions in the power states, and allows it to respond to various conditions such as full power, power managed for efficiency, suspend, and sleep states.

- [MNCRS Java Application Level Power Management Specification](#)
- [Power Management API Specification](#)

Communication API

The Communication API extension to the PersonalJava platform is used by applications to access serial or parallel ports on the device. If the device does not have application accessible ports, this API is not required to be present. For example, if the device has a modem or printer that is directly available to applications, it would be accessed through the javax.comm package.

The types of ports supported are displayed in the following table.

Port Description	Port Name	Port Type
Main RS232 Serial Port	COM1	PORT_SERIAL
Emulated Serial Infrared Port	IrCOMM	PORT_SERIAL

Devices that have an infrared port should implement the serial emulation of the Infrared Data Association standard [IrCOMM: Serial and Parallel Port Emulation](#).

- [Communications API 2.0 specification](#)

Secure Sockets Layer (SSL)

The optional SSL package provides secure communication over TCP/IP sockets. It contains support for both client side use of secure sockets, as well as the server socket interfaces for supporting certificate based authentication.

- [SSL Sockets API Specification](#)

PersonalJava™ Optional Packages

Optional features are not required to be supported in every PersonalJava™ Application Environment(PJAE). The choice of whether to support the feature is left to the PJAE implementor. However, if the implementor elects to support a given feature, then the implementation must support it completely and retain exactly the same API as its counterpart in the JDK™ platform. An optional feature that is excluded from an implementation is called an unsupported optional feature.

Examples of optional features include the following:

- `java.math` is an optional package.
- `java.awt.CheckboxMenuItem` is an optional class.
- `java.awt.Toolkit.getPrintJob` is an optional method.

The PersonalJava™ platform specification provides the following optional packages and classes as well as network protocols that may apply to the implementation of the JavaPhone™ API:

- Optional packages
- Modified packages
- Optional classes
- Network protocols

Optional Packages

Component	Required	Optional	Comment
<code>java.math</code>		X	If the optional <code>java.security</code> , <code>java.sql</code> package or the optional <u>SSL Java standard extension</u> are supported, then <code>java.math</code> is required, otherwise it is optional.
<code>java.rmi</code> <code>java.rmi.dgc</code> <code>java.rmi.registry</code> <code>java.rmi.server</code>		X	When the optional <code>java.rmi</code> package is implemented each of the sub-packages must also be supported.
<code>java.security</code> <code>java.security.interfaces</code>		X	When the optional <code>javax.security</code> package is implemented <code>java.security.interfaces</code> must also be supported.

Modified Packages

Component	Required	Optional	Comment
<code>java.awt</code>	X		See the PersonalJava specification for details about the modified <code>java.awt</code> classes.
<code>java.io</code>	X		The <code>java.io</code> package is required except for the optional classes described below.

Optional Classes

Component	Required	Optional	Comment
<code>java.io.File</code> <code>java.io.FileInputStream</code> <code>java.io.FileNotFoundException</code> <code>java.io.FileOutputStream</code> <code>java.io.FileReader</code> <code>java.io.FileWriter</code> <code>java.io FilenameFilter</code> <code>java.io.RandomAccessFile</code>		X	This set of classes is optional as a group. When a user-visible file system is included on a platform <code>java.awt.FileDialog</code> and <code>java.awt.peer.FileDialog</code> must also be supported.
<code>java.text.resources</code>		X	A complete implementation includes support for <code>DateFormatZoneData</code>, <code>LocaleData</code>, <code>LocaleElements</code> classes and at least one pair of locale specific classes, e.g. <code>DateFormatZoneData_en_US</code> and <code>LocaleElements_en_US</code>
<code>java.util.zip</code>		X	See the PersonalJava specification for details of the modifications to the <code>java.util.zip</code> package

Networking Protocols

Protocol	Required	Optional	Comment
http	X		The <code>http</code> protocol scheme provides basic support for WWW aware applications. The PersonalJava specification requires at least the 1.0 version of the HTTP protocol to be supported. (See IETF RFC1945)
https		X	The <code>https</code> protocol scheme layers HTTP requests over a secure socket connection using the standard <code>java.net.URL</code> class and an implementation specific content handler. The PersonalJava specification requires at least version 3.0 support for SSL applications.
mailto	X		The <code>mailto</code> protocol scheme provides client access to mail transfer agent protocol requests. (See IETF RFC2368)
file		X	The <code>file</code> scheme provides for local file system access. (See IETF RFC1738)

JavaPhone™ Specification

JavaPhone Specification 1.0	
<u>javax.ce.install</u>	The Install package manages applications and standard extensions present in a Java application environment.
<u>javax.net.datagram</u>	The Network Datagram API allows program to program communication by means of sending and receiving datagrams.
<u>javax.pim.addressbook</u>	The AddressBook package defines the objects used to access, add, delete and update the contact information in a contact database.
<u>javax.pim.calendar</u>	The Calendar package defines objects to access, add, delete and update the calendar event information in a Calendar database.
<u>javax.pim.database</u>	The Database package defines the objects used to access Addressbook and Calendar information.
<u>javax.pim.userprofile</u>	The UserProfile package defines objects to get and set the current user contact information, which is returned in the same format as the AddressBook package.
<u>javax.power.management</u>	The Power Management package allows an application to monitor and respond to changes in the power states of the system.
<u>javax.power.monitor</u>	The Power Monitor package allows an application to monitor and respond to changes in the power level of the system.
<u>javax.telephony</u>	The Java Telephony API (JTAPI) is a portable, object-oriented application programming interface for Java-based computer-telephony applications.
<u>javax.telephony.callcontrol</u>	The JTAPI advanced Call Control package provides more detailed information about the core call model and supplies an extended set of states for the Connection and TerminalConnection objects.
<u>javax.telephony.callcontrol.capabilities</u>	The JTAPI Call Control Capabilities package provides the interfaces that inform the an application about which features were included in the current JTAPI implementation and which features are currently valid throughout a call transaction.
<u>javax.telephony.callcontrol.events</u>	The JTAPI Call Control Events package defines the specific Call Control state transition events.

<u>javax.telephony.capabilities</u>	The JTAPI Core Capabilities package provides an interface to query the current JTAPI implementation to see which features are currently accessible.
<u>javax.telephony.events</u>	The JTAPI Core Events package defines the core call model state transitions in setting up and monitoring call progress.
<u>javax.telephony.mobile</u>	The JTAPI Mobile package extends the core capabilities of a JTAPI implementation with capabilities specific to mobile networks.
<u>javax.telephony.phone</u>	The JTAPI Phone package provides objects that model the physical characteristics of the terminal, such as, speakerphone, microphone, display, buttons, ringer, handset, and lamp.
<u>javax.telephony.phone.capabilities</u>	The JTAPI Phone Capabilities package defines the optional features that may be present as part of the JTAPI implementation.
<u>javax.telephony.phone.events</u>	The JTAPI Phone Events package defines the specific event transitions associated with the physical terminal characteristics.

Hierarchy For All Packages

Package Hierarchies:

javax.ce.install, javax.net.datagram, javax.pim.addressbook, javax.pim.calendar, javax.pim.database,
javax.pim.userprofile, javax.power.management, javax.power.monitor, javax.telephony,
javax.telephony.callcontrol, javax.telephony.callcontrol.capabilities,
javax.telephony.callcontrol.events, javax.telephony.capabilities, javax.telephony.events,
javax.telephony.mobile, javax.telephony.phone, javax.telephony.phone.capabilities,
javax.telephony.phone.events

Class Hierarchy

- class java.lang.Object
 - class javax.pim.calendar.**CalendarDatabase** (implements javax.pim.database.Database)
 - class javax.telephony.callcontrol.**CallControlForwarding**
 - class javax.pim.addressbook.**ContactDatabase** (implements javax.pim.database.Database)
 - class javax.net.datagram.**Datagram**
 - class javax.net.datagram.**DatagramNameService**
 - class javax.net.datagram.**DatagramService**
 - class javax.net.datagram.**DatagramServiceFactory**
 - class java.util.EventObject (implements java.io.Serializable)
 - class javax.pim.database.**DatabaseEvent**
 - class javax.power.management.**PowerStateEvent**
 - class javax.ce.install.**Installer**
 - class javax.ce.install.**InstallInfo**
 - class javax.pim.database.**Item**
 - class javax.pim.calendar.**CalendarEntry**
 - class javax.pim.calendar.**CalendarToDo**
 - class javax.pim.addressbook.**ContactCard**
 - class javax.pim.addressbook.**ContactGroup**
 - class javax.pim.addressbook.**ContactTemplate**
 - class javax.pim.database.**ItemField**
 - class javax.pim.database.**AggregateField**
 - class javax.telephony.**JtapiPeerFactory**
 - class javax.pim.database.**Parameter**
 - class javax.power.management.**PowerManager**
 - class javax.power.monitor.**PowerMonitor**
 - class javax.power.management.**PowerState**
 - class javax.power.monitor.**PowerWarningType**
 - class javax.pim.calendar.**Repeat**
 - class javax.pim.calendar.**RepeatRule**

- class java.lang.Throwable (implements java.io.Serializable)
 - class java.lang.Exception
 - class javax.pim.database.**DatabaseException**
 - class javax.pim.database.**DatabaseUpdateException**
 - class javax.telephony.**InvalidArgumentException**
 - class javax.telephony.**InvalidPartyException**
 - class javax.telephony.**InvalidStateException**
 - class java.io.IOException
 - class javax.net.datagram.**AddressNotSupportedException**
 - class javax.net.datagram.**IntermittentNetworkException**
 - class javax.net.datagram.**MalformedAddressException**
 - class javax.telephony.**JtapiPeerUnavailableException**
 - class javax.telephony.**MethodNotSupportedException**
 - class javax.power.management.**PowerManagerException**
 - class javax.telephony.**PrivilegeViolationException**
 - class javax.telephony.**ResourceUnavailableException**
 - class java.lang.RuntimeException
 - class javax.telephony.**PlatformException**
 - class javax.telephony.**ProviderUnavailableException**
- class javax.pim.userprofile.**UserProfile**

Interface Hierarchy

- interface javax.telephony.**Address**
 - interface javax.telephony.callcontrol.**CallControlAddress**
 - interface javax.telephony.mobile.**MobileAddress**
- interface javax.net.datagram.**Address**
- interface javax.telephony.capabilities.**AddressCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlAddressCapabilities**
- interface javax.telephony.**AddressObserver**
 - interface javax.telephony.callcontrol.**CallControlAddressObserver**
- interface javax.telephony.**Call**
 - interface javax.telephony.callcontrol.**CallControlCall**
- interface javax.telephony.capabilities.**CallCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlCallCapabilities**
- interface javax.telephony.**CallObserver**
 - interface javax.telephony.callcontrol.**CallControlCallObserver**
- interface javax.telephony.phone.**Component**
 - interface javax.telephony.phone.**PhoneButton**
 - interface javax.telephony.phone.**PhoneDisplay**
 - interface javax.telephony.phone.**PhoneGraphicDisplay**
 - interface javax.telephony.phone.**PhoneHookswitch**
 - interface javax.telephony.phone.**PhoneLamp**

- interface javax.telephony.phone.**PhoneMicrophone**
- interface javax.telephony.phone.**PhoneRinger**
- interface javax.telephony.phone.**PhoneSpeaker**
- interface javax.telephony.phone.capabilities.**ComponentCapabilities**
- interface javax.telephony.phone.**ComponentGroup**
- interface javax.telephony.phone.capabilities.**ComponentGroupCapabilities**
- interface javax.telephony.**Connection**
 - interface javax.telephony.callcontrol.**CallControlConnection**
- interface javax.telephony.capabilities.**ConnectionCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlConnectionCapabilities**
- interface javax.pim.database.**Database**
- interface javax.pim.database.**DatabaseFactory**
- interface javax.telephony.events.**Ev**
 - interface javax.telephony.events.**AddrEv**
 - interface javax.telephony.events.**AddrObservationEndedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlAddrEv**(also extends javax.telephony.callcontrol.events.**CallCtlEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlAddrDoNotDisturbEv**
 - interface javax.telephony.callcontrol.events.**CallCtlAddrForwardEv**
 - interface javax.telephony.callcontrol.events.**CallCtlAddrMessageWaitingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlEv**
 - interface javax.telephony.callcontrol.events.**CallCtlAddrEv**(also extends javax.telephony.events.**AddrEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlAddrDoNotDisturbEv**
 - interface javax.telephony.callcontrol.events.**CallCtlAddrForwardEv**
 - interface javax.telephony.callcontrol.events.**CallCtlAddrMessageWaitingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlCallEv**(also extends javax.telephony.events.**CallEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnEv**(also extends javax.telephony.events.**ConnEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDialingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDisconnectedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnEstablishedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnFailedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnInitiatedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkReachedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnOfferedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnQueuedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnUnknownEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnEv**(also extends javax.telephony.events.**TermConnEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnBridgedEv**

- interface javax.telephony.callcontrol.events.**CallCtlTermConnDroppedEv**
- interface javax.telephony.callcontrol.events.**CallCtlTermConnHeldEv**
- interface javax.telephony.callcontrol.events.**CallCtlTermConnInUseEv**
- interface javax.telephony.callcontrol.events.**CallCtlTermConnRingingEv**
- interface javax.telephony.callcontrol.events.**CallCtlTermConnTalkingEv**
- interface javax.telephony.callcontrol.events.**CallCtlTermConnUnknownEv**
- interface javax.telephony.callcontrol.events.**CallCtlTermEv**(also extends javax.telephony.events.**TermEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermDoNotDisturbEv**
- interface javax.telephony.events.**CallEv**
 - interface javax.telephony.events.**CallActiveEv**
 - interface javax.telephony.callcontrol.events.**CallCtlCallEv**(also extends javax.telephony.callcontrol.events.**CallCtlEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnEv**(also extends javax.telephony.events.**ConnEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDialingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDisconnectedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnEstablishedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnFailedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnInitiatedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnNetworkReachedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnOfferedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnQueuedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnUnknownEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnEv**(also extends javax.telephony.events.**TermConnEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnBridgedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnDroppedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnHeldEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnInUseEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnRingingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnTalkingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnUnknownEv**
 - interface javax.telephony.events.**CallInvalidEv**
 - interface javax.telephony.events.**CallObservationEndedEv**
 - interface javax.telephony.events.**ConnEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnEv**(also extends javax.telephony.callcontrol.events.**CallCtlCallEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlConnAlertingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDialingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnDisconnectedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlConnEstablishedEv**

- interface javax.telephony.callcontrol.events.**CallCtlConnFailedEv**
- interface javax.telephony.callcontrol.events.**CallCtlConnInitiatedEv**
- interface javax.telephony.callcontrol.events.**CallCtlConnNetworkAlertingEv**
- interface javax.telephony.callcontrol.events.**CallCtlConnNetworkReachedEv**
- interface javax.telephony.callcontrol.events.**CallCtlConnOfferedEv**
- interface javax.telephony.callcontrol.events.**CallCtlConnQueuedEv**
- interface javax.telephony.callcontrol.events.**CallCtlConnUnknownEv**
- interface javax.telephony.events.**ConnAlertingEv**
- interface javax.telephony.events.**ConnConnectedEv**
- interface javax.telephony.events.**ConnCreatedEv**
- interface javax.telephony.events.**ConnDisconnectedEv**
- interface javax.telephony.events.**ConnFailedEv**
- interface javax.telephony.events.**ConnInProgressEv**
- interface javax.telephony.events.**ConnUnknownEv**
- interface javax.telephony.events.**TermConnEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnEv**(also extends javax.telephony.callcontrol.events.**CallCtlCallEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnBridgedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnDroppedEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnHeldEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnInUseEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnRingingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnTalkingEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermConnUnknownEv**
 - interface javax.telephony.events.**TermConnActiveEv**
 - interface javax.telephony.events.**TermConnCreatedEv**
 - interface javax.telephony.events.**TermConnDroppedEv**
 - interface javax.telephony.events.**TermConnPassiveEv**
 - interface javax.telephony.events.**TermConnRingingEv**
 - interface javax.telephony.events.**TermConnUnknownEv**
- interface javax.telephony.phone.events.**PhoneEv**
 - interface javax.telephony.phone.events.**PhoneTermEv**(also extends javax.telephony.events.**TermEv**)
 - interface javax.telephony.phone.events.**ButtonInfoEv**
 - interface javax.telephony.phone.events.**ButtonPressEv**
 - interface javax.telephony.phone.events.**DisplayUpdateEv**
 - interface javax.telephony.phone.events.**HookswitchStateEv**
 - interface javax.telephony.phone.events.**LampModeEv**
 - interface javax.telephony.phone.events.**MicrophoneGainEv**
 - interface javax.telephony.phone.events.**RingerPatternEv**
 - interface javax.telephony.phone.events.**RingerVolumeEv**
 - interface javax.telephony.phone.events.**SpeakerVolumeEv**
- interface javax.telephony.events.**ProvEv**

- interface javax.telephony.events.**ProvInServiceEv**
- interface javax.telephony.events.**ProvObservationEndedEv**
- interface javax.telephony.events.**ProvOutOfServiceEv**
- interface javax.telephony.events.**ProvShutdownEv**
- interface javax.telephony.events.**TermEv**
 - interface javax.telephony.callcontrol.events.**CallCtlTermEv**(also extends javax.telephony.callcontrol.events.**CallCtlEv**)
 - interface javax.telephony.callcontrol.events.**CallCtlTermDoNotDisturbEv**
 - interface javax.telephony.phone.events.**PhoneTermEv**(also extends javax.telephony.phone.events.**PhoneEv**)
 - interface javax.telephony.phone.events.**ButtonInfoEv**
 - interface javax.telephony.phone.events.**ButtonPressEv**
 - interface javax.telephony.phone.events.**DisplayUpdateEv**
 - interface javax.telephony.phone.events.**HookswitchStateEv**
 - interface javax.telephony.phone.events.**LampModeEv**
 - interface javax.telephony.phone.events.**MicrophoneGainEv**
 - interface javax.telephony.phone.events.**RingerPatternEv**
 - interface javax.telephony.phone.events.**RingerVolumeEv**
 - interface javax.telephony.phone.events.**SpeakerVolumeEv**
 - interface javax.telephony.events.**TermObservationEndedEv**
- interface javax.telephony.**Event**
 - interface javax.telephony.**AddressEvent**
 - interface javax.telephony.**CallEvent**
 - interface javax.telephony.**ConnectionEvent**
 - interface javax.telephony.**TerminalConnectionEvent**
 - interface javax.telephony.**MetaEvent**
 - interface javax.telephony.**MultiCallMetaEvent**
 - interface javax.telephony.**SingleCallMetaEvent**
 - interface javax.telephony.mobile.**MobileRadioEvent**
 - interface javax.telephony.**ProviderEvent**
 - interface javax.telephony.mobile.**MobileProviderEvent**
 - interface javax.telephony.**TerminalEvent**
- interface java.util.EventListener
 - interface javax.telephony.**AddressListener**
 - interface javax.telephony.**CallListener**
 - interface javax.telephony.**ConnectionListener**
 - interface javax.telephony.**TerminalConnectionListener**
 - interface javax.pim.database.**DatabaseListener**
 - interface javax.power.monitor.**PowerMonitorListener**
 - interface javax.power.management.**PowerStateListener**
 - interface javax.telephony.**ProviderListener**
 - interface javax.telephony.mobile.**MobileProviderListener**
 - interface javax.telephony.**TerminalListener**

- interface javax.pim.database.**Iterator**
 - interface javax.telephony.**JtapiPeer**
 - interface javax.telephony.mobile.**MobileNetwork**
 - interface javax.telephony.mobile.**MobileRadio**
 - interface javax.telephony.mobile.**MobileRadioListener**
 - interface javax.telephony.**Provider**
 - interface javax.telephony.mobile.**MobileProvider**
 - interface javax.telephony.mobile.**NetworkSelection**
 - interface javax.telephony.capabilities.**ProviderCapabilities**
 - interface javax.telephony.**ProviderObserver**
 - interface javax.telephony.**Terminal**
 - interface javax.telephony.callcontrol.**CallControlTerminal**
 - interface javax.telephony.mobile.**MobileTerminal**
 - interface javax.telephony.phone.**PhoneTerminal**
 - interface javax.telephony.capabilities.**TerminalCapabilities**
 - interface javax.telephony.callcontrol.capabilities.**CallControlTerminalCapabilities**
 - interface javax.telephony.**TerminalConnection**
 - interface javax.telephony.callcontrol.**CallControlTerminalConnection**
 - interface javax.telephony.capabilities.**TerminalConnectionCapabilities**
 - interface
javax.telephony.callcontrol.capabilities.**CallControlTerminalConnectionCapabilities**
 - interface javax.telephony.**TerminalObserver**
 - interface javax.telephony.callcontrol.**CallControlTerminalObserver**
 - interface javax.telephony.phone.**PhoneTerminalObserver**
 - interface javax.pim.userprofile.**UserProfileImpl**
-
-

JavaPhone Specification

Table of Contents

JavaPhone Specification	1
JavaPhone™ API Specification	1
JavaPhone™ Overview	2
PersonalJava Platform and JavaPhone	3
JavaPhone Interfaces	3
Optional PersonalJava Interfaces	4
Security Recommendations for JavaPhone Interfaces	4
Java™ Telephony API (JTAPI) Core Package	5
JTAPI Package Specifications	5
JTAPI Core Package	5
JTAPI Call Control Package	6
JTAPI Phone Package	6
JTAPI Mobile Package	6
JTAPI Mobile core interfaces	7
AddressBook	7
User Profile	8
Calendar	8
Network Datagram	8
Install	9
Power Monitor	9
Power Management	10
Communication API	10
Secure Sockets Layer (SSL)	10
JavaPhone Platform Specification	11
PersonalJava™ Optional Packages	11
Optional Packages	11
Modified Packages	12
Optional Classes	12
Networking Protocols	12
Overview	14
JavaPhone™ Specification	14
Class Hierarchy	16
Hierarchy For All Packages	16
Class Hierarchy	16
Interface Hierarchy	17